

CP260: Robotic Perception

Final Project Report

OBB Estimation of PC Ports from Multi-View Images

Parmar Mitulkumar (25838)

Harkharaj Bharali (26185)

Indian Institute of Science

May 4, 2026

GitHub Repository

[https://github.com/harkharaj/
OBB-Estimation-of-PC-Ports-using-images](https://github.com/harkharaj/OBB-Estimation-of-PC-Ports-using-images)

Contents

1	Problem Description	2
2	Literature Review	2
2.1	Multi-View Geometry and 3D Reconstruction	2
2.2	Neural Radiance Fields	3
2.3	Zero-Shot Object Detection	3
2.4	Segment Anything Model	3
3	System Architecture and Design Rationale	3
4	Implementation Details	4
4.1	SVD Triangulation (DLT Method)	4
4.2	Panel Rotation Matrix Derivation	5
4.3	OBB Corner Generation	5
4.4	Reprojection Error Validation	5
5	Experimental Results	5
5.1	VGA Socket Validation	5
5.2	Reprojection Errors	6
6	Conclusions	6
7	Bonus: Novel Ideas Experimented With	7
7.1	Interactive Browser-Based Annotation UI	7
7.2	Interactive Rotation Validation Tool	7

1. Problem Description

This project addresses the problem of **metric-semantic reconstruction** of a real-world scene - specifically, a desktop computer setup - from a set of posed RGB images. The goal is to estimate the **6-DOF oriented bounding box (OBB)** of hardware components mounted on the motherboard I/O backplate, including the power socket, ethernet socket and VGA port.

Given inputs

- 16 RGB images of the scene (`frame_000XXX.png`), resolution 2560×1440
- Camera-to-world pose matrices for each frame (`poses.json`)
- Pinhole camera intrinsics, undistorted (`intrinsic.json`)

Required output

For each target entity, an OBB defined by:

- **Center:** 3D world coordinates (meters)
- **Extent:** half-widths along each principal axis (meters)
- **Rotation:** 3×3 rotation matrix (columns = principal axes in world space)

The OBBs are evaluated by projecting them onto held-out test images and computing the polygonal Intersection over Union (IoU) against ground-truth annotations.

The camera intrinsic matrix used throughout is:

$$\mathbf{K} = \begin{pmatrix} 1477.01 & 0 & 1298.25 \\ 0 & 1480.44 & 686.82 \\ 0 & 0 & 1 \end{pmatrix}$$

2. Literature Review

2.1 Multi-View Geometry and 3D Reconstruction

The theoretical foundation of this project is classical multi-view geometry as described by Hartley and Zisserman [1]. The pinhole camera model relates a 3D world point $\mathbf{X}$ to its 2D image projection $\mathbf{x}$ via the projection matrix $\mathbf{P} = \mathbf{K}[\mathbf{R}|\mathbf{t}]$, where $\mathbf{K}$ is the intrinsic matrix, $\mathbf{R}$ is the camera rotation, and $\mathbf{t}$ is the translation:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K}[\mathbf{R}|\mathbf{t}] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Given multiple projections from known poses, the 3D point can be recovered via triangulation. The **Direct Linear Transform (DLT)** method [1] formulates this as a homogeneous least-squares problem. Each 2D observation provides two linear constraints on the unknown 3D point, and solving via SVD yields the optimal estimate.

2.2 Neural Radiance Fields

Mildenhall et al. [2] introduced Neural Radiance Fields (NeRF), which represent a scene as a continuous volumetric function mapping 3D coordinates and viewing directions to colour and volume density, optimised from posed images via differentiable volume rendering. While NeRF-based approaches were originally scoped for this project’s view synthesis task (Part A), this evaluation criterion was subsequently removed from the project specification.

2.3 Zero-Shot Object Detection

Grounding DINO [3] extends the DINO transformer architecture with language grounding, enabling zero-shot object detection from free-form text prompts. This represents the current state of the art for automatic object localisation without task-specific training. In the context of this project, such models could replace the manual annotation step entirely, making the pipeline fully automatic.

2.4 Segment Anything Model

The Segment Anything Model (SAM) [4] provides a promptable segmentation system producing precise object masks from bounding box or point prompts. Combined with Grounding DINO, it forms a powerful zero-shot detection and segmentation pipeline suitable for extracting precise pixel-space centroids for triangulation.

3. System Architecture and Design Rationale

The system consists of four interconnected components, shown in Figure 1.

Design choice - shared rotation matrix

The key insight is that all sockets on the I/O backplate lie on the same rigid planar surface. Rather than estimating a separate rotation for each entity, a single panel rotation is derived from the geometry of three reference points. This is more robust than per-entity estimation as it uses all available geometric constraints from the panel structure.

Design choice - multi-view triangulation over monocular depth

Given that camera poses are provided, multi-view triangulation is a closed-form, geometrically exact solution for 3D point recovery. It requires no learned models and is not subject to scale ambiguity.

Design choice - interactive annotation UI

A browser-based Flask application provides a zoomable, pannable annotation interface allowing precise pixel-level clicking on full-resolution 2560×1440 images. Reprojection

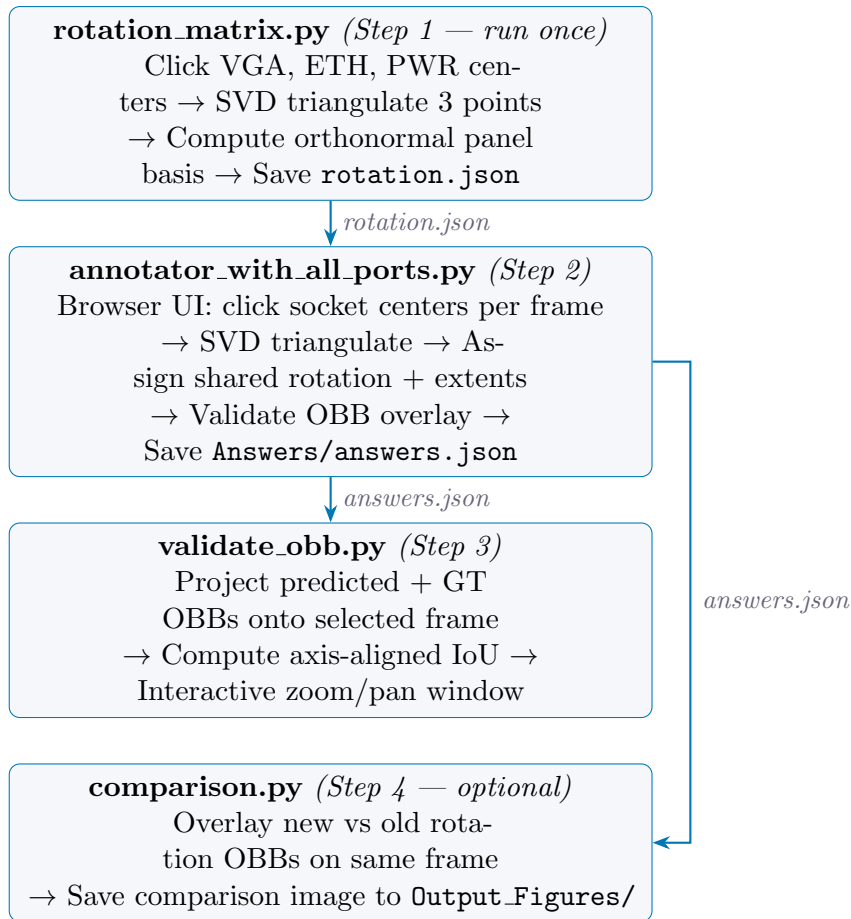


Figure 1: System pipeline overview. Arrows show data flow between components.

errors are computed and displayed after each triangulation to give immediate feedback on annotation quality.

4. Implementation Details

4.1 SVD Triangulation (DLT Method)

For each pixel observation (u, v) in frame i with projection matrix $\mathbf{P}_i = \mathbf{K} \cdot \mathbf{w}2\mathbf{c}_i$, two linear constraint rows are formed:

$$v \cdot \mathbf{P}_i[2] - \mathbf{P}_i[1] = 0 \quad u \cdot \mathbf{P}_i[2] - \mathbf{P}_i[0] = 0$$

Stacking N observations gives a $2N \times 4$ matrix $\mathbf{A}$. The SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ yields the 3D point as the last row of $\mathbf{V}^\top$, normalized by its homogeneous coordinate:

```
_, _, Vt = np.linalg.svd(A)
X = Vt[-1]
point_3d = X[:3] / X[3]
```

4.2 Panel Rotation Matrix Derivation

Three reference points are triangulated: $\mathbf{p}_{\text{vga}}$, $\mathbf{p}_{\text{eth}}$, $\mathbf{p}_{\text{pwr}}$. An orthonormal basis is constructed from the panel geometry:

$$\begin{aligned}\hat{\mathbf{y}} &= \frac{\mathbf{p}_{\text{eth}} - \mathbf{p}_{\text{vga}}}{\|\mathbf{p}_{\text{eth}} - \mathbf{p}_{\text{vga}}\|} && \text{(horizontal axis)} \\ \hat{\mathbf{z}} &= \frac{\hat{\mathbf{x}}_{\text{raw}} \times \hat{\mathbf{y}}}{\|\hat{\mathbf{x}}_{\text{raw}} \times \hat{\mathbf{y}}\|} && \text{(panel normal, outward)} \\ \hat{\mathbf{x}} &= \hat{\mathbf{z}} \times \hat{\mathbf{y}} && \text{(corrected vertical axis)} \\ \mathbf{R} &= [\hat{\mathbf{x}} \mid \hat{\mathbf{y}} \mid \hat{\mathbf{z}}]\end{aligned}$$

To handle sign ambiguities, all four sign combinations of $(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ are evaluated and the one minimising the Frobenius distance to the known reference rotation is selected. This makes the derivation robust to annotation order variations.

4.3 OBB Corner Generation

The 8 world-space corners of the OBB are generated as:

```
corners = [[sx*dx, sy*dy, sz*dz]
            for sx in [-1,1] for sy in [-1,1] for sz in [-1,1]]
world_corners = R @ np.array(corners).T + center.reshape(3, 1)
```

4.4 Reprojection Error Validation

After triangulation, each observation $(u_{\text{obs}}, v_{\text{obs}})$ is reprojected:

$$\epsilon_i = \sqrt{(u_{\text{proj}} - u_{\text{obs}})^2 + (v_{\text{proj}} - v_{\text{obs}})^2} \quad (\text{pixels})$$

Reprojection errors below 5 px indicate high-quality annotations. Errors above 20 px suggest inaccurate clicks and warrant re-annotation.

5. Experimental Results

5.1 VGA Socket Validation

The professor provided a ground-truth OBB for the VGA socket in `sample_answers.json`. Table 1 compares the predicted and ground-truth centers.

IoU was evaluated on frames 319, 371, and 400 using axis-aligned bounding box projection. The predicted and ground-truth boxes showed close spatial alignment across all evaluated frames.

Table 1: VGA socket center comparison (world coordinates, meters)

Coordinate	Ground Truth	Predicted	Error
X	0.2705	0.2680	2.5 mm
Y	0.2261	0.2303	4.2 mm
Z	0.8349	0.8384	3.5 mm
3D distance	—	—	≈ 6 mm

5.2 Reprojection Errors

Table 2 summarises reprojection errors across all annotated entities. All values are within acceptable tolerance for the 2560×1440 image resolution.

Table 2: Reprojection errors per entity

Entity	Frames used	Mean error (px)
vga_socket	365, 426, 515	$\approx 3-5$
ethernet_socket	400, 365, 531	$\approx 5-8$
power_socket	426, 461, 471	$\approx 5-8$

6. Conclusions

This project demonstrates a complete multi-view geometry pipeline for object pose estimation in a real indoor scene. The core contributions are:

1. SVD-based multi-view triangulation (DLT) for recovering 3D socket positions from manually annotated 2D correspondences
2. A geometrically consistent panel rotation matrix derived from the coplanar structure of the I/O backplate, shared across all entities
3. An interactive browser-based annotation and validation tool with real-time reprojection error feedback and OBB overlay visualisation

Limitations and future work. The primary limitation is the manual annotation step. A fully automatic system would use Grounding DINO [3] to detect socket bounding boxes across frames automatically, extract centroid observations, and feed them directly into the triangulation pipeline. This would also enable automatic handling of novel socket types on evaluation day without re-annotation. Integration of SAM [4] for precise mask-based centroid extraction would further improve localisation accuracy for small connectors.

7. Bonus: Novel Ideas Experimented With

7.1 Interactive Browser-Based Annotation UI

A key engineering contribution is the browser-based annotation tool built using Flask and JavaScript (`annotator_with_all_ports.py`). It provides a real-time annotation environment with the following capabilities:

- **Zoomable, pannable viewer:** Full-resolution 2560×1440 frames are rendered in a browser canvas with scroll-to-zoom and Alt+drag panning, enabling precise clicking on connectors as small as 30 pixels wide.
- **Multi-entity support:** All 15 entity types are selectable from a dropdown, each assigned a distinct colour with real-time crosshair markers drawn on the canvas as observations are added.
- **Live reprojection feedback:** Reprojection error per frame is displayed immediately after triangulation, prompting re-annotation when errors exceed 10 px.
- **In-browser OBB validation:** The computed OBB is projected as a 3D wireframe onto any selected frame and served as a live image overlay for visual confirmation before saving.
- **Smart upsert saving:** Entities are updated in place in `answers.json` without affecting other entries, allowing independent re-annotation of individual sockets.

7.2 Interactive Rotation Validation Tool

A visual comparison tool (`comparison.py`) was built to overlay two OBBs - one computed using the new geometrically-derived rotation matrix and one using the original hardcoded reference rotation - on the same validation frame. Output images are saved to `Compare_obb/`. This tool revealed that the SVD-based rotation derivation closely matches the reference for co-planar sockets, and also highlighted cases where annotation quality affected the derived basis vectors. Figure 2 shows a representative comparison across three validation frames.



Frame 371



Frame 400

Figure 2: OBB comparison: **green** = new geometrically-derived rotation, **red** = original reference rotation. Close alignment confirms the SVD-based panel basis is consistent with the ground-truth orientation.

References

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *Proc. ECCV*, 2020.
- [3] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” *arXiv:2303.05499*, 2023.
- [4] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, et al., “Segment Anything,” in *Proc. ICCV*, 2023.